SIMATS ENGINEERING
ASSESSMENT TOOL 2

Course Name:	Cryptography and Network Security
Course Code:	CSA51
Course Outcome Covered:	*CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)*
Scenario-Based:	20%
Total Marks:	50
Student Name:	Vaddi Bharathwaj Reddy
Reg. No.:	192372213

Annexure A – Scenario-Based

Questions Attempted: As permitted, 2 of the 5 scenarios below have been attempted in full — Question 2 (Choosing an Encryption Algorithm for Banking System) and Question 5 (Cryptography for IoT Devices).

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and	Generally clear with minor issues	Somewhat unclear or poorly	Disorganized and difficult to

		logically presented		structured	understand

Question 2: Choosing an Encryption Algorithm for Banking System

Scenario: A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Most suitable algorithm for this scenario

Blowfish is the most suitable choice for this banking scenario among the three options, provided a sufficiently long key (e.g., 128 bits or more) is used.

Comparative analysis

Algorithm	Key Size	Security Strength	Performance / Efficiency
DES	56 bits	Low — broken by brute force in hours with modern hardware; considered cryptographically weak today.	Fast, but security is inadequate for financial data.
3-DES	168 bits (effective ~112-bit security)	Moderate-to-high — significantly stronger than DES, still used in legacy banking/payment systems (e.g., EMV).	About 3× slower than DES due to triple encryption; noticeably heavier than Blowfish.
Blowfish	32–448 bits (variable)	High — no practical cryptanalytic break for full-round Blowfish; flexible key length allows tuning security level.	Fast software performance, low memory footprint, well suited to moderate-performance requirements.

B. Justification based on security strength and efficiency

- Security: DES's 56-bit key is now trivially breakable by brute force and is unacceptable for financial data; it should be ruled out immediately for a banking system.
- 3-DES improves security considerably over DES (effective strength ~112 bits) and is still deployed in some legacy payment systems, but it inherits DES's small 64-bit block size, which creates a theoretical risk of block-collision attacks (e.g., the Sweet32 attack) when large volumes of data are encrypted under one key — a real concern for a high-transaction-volume banking application.
- Blowfish uses a 64-bit block size as well, but its variable key length (up to 448 bits) allows the bank to configure a strong security margin, and no practical cryptanalytic attack exists against full-round Blowfish.
- Performance: Blowfish is significantly faster in software than 3-DES (which performs three full DES encryptions per block) and has a smaller memory footprint, directly satisfying the "moderate performance" requirement of the scenario.
- Overall trade-off: Blowfish offers the best balance of strong, tunable security and efficient performance for this scenario, making it preferable to the outdated DES and the comparatively slower 3-DES. (In a modern greenfield banking deployment, AES or Twofish would typically be preferred over all three, but restricted to the given options, Blowfish is the correct recommendation.)

Question 5: Cryptography for IoT Devices

Scenario: A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analysis of IoT device constraints

Constraint	Impact on Cryptographic Choice
Processing power	IoT devices typically use low-power microcontrollers with limited CPU cycles, making large modular-exponentiation operations (as required by RSA) expensive.
Memory / storage	Limited RAM and flash storage make it costly to store and process large keys (RSA needs ~1024–2048-bit keys) and large intermediate computation buffers.
Battery / energy	Devices are often battery-powered; every CPU cycle spent on cryptographic computation directly reduces battery life.
Bandwidth	Constrained networks (e.g., Zigbee, BLE, LoRa) have low data rates, so larger keys and certificates (as required by RSA) increase transmission overhead and latency.
Cost	IoT devices are produced at scale and must remain cheap, limiting the hardware (e.g., crypto accelerators) available for expensive public-key operations.

B. Recommended approach and justification

Recommendation: Elliptic Curve Cryptography (ECC) is the more suitable cryptographic approach for this smart home IoT scenario.

- Equivalent security at much smaller key sizes: a 160-bit ECC key provides security roughly comparable to a 1024-bit RSA key, and a 256-bit ECC key is comparable to 3072-bit RSA — dramatically reducing the computation, memory, and bandwidth needed to achieve the same security level.
- Lower computational cost: ECC's point-multiplication operations are far less expensive than RSA's large modular exponentiations, translating directly into faster operations and lower energy consumption on constrained microcontrollers.
- Reduced power consumption: because ECC needs fewer CPU cycles to achieve equivalent security, it extends battery life — critical for smart home sensors and actuators that may run for years on a small battery.
- Smaller memory and bandwidth footprint: shorter keys and certificates mean less flash/RAM usage on the device and smaller payloads exchanged over constrained wireless links (Zigbee, BLE, LoRa), reducing both latency and energy spent on radio transmission.
- Practical adoption: ECC-based schemes (e.g., ECDH for key exchange, ECDSA for signatures) are already the standard recommendation in constrained-device security frameworks and IoT security protocols (e.g., TLS with ECC cipher suites), confirming their suitability here.

Conclusion: Given the smart home system's tight processing, memory, energy, and bandwidth constraints, ECC provides the same security guarantees as RSA at a fraction of the resource cost, making it the clearly preferable cryptographic approach for this IoT scenario.